```
int readSensor() {
  int sensorValue = 0;
  for (int i=0; i<5; i++) {
    sensorValue = sensorValue + analogRead(A0);
    delay(10);
  }
  sensorValue = map(sensorValue, 0, 5115, 0, 255);
  return sensorValue;
}
```

In this function, we first create a new local variable called `sensorValue`. Then we start a `for` loop that will loop five times. Each time through the loop, it will add the current sensor reading to the running total and pause for 10 milliseconds just to get a better average. Once we have our new total, we will use the `map()` function to map our reading to a value with a new range of 0–255. That odd number 5115 is equal to 5 * 1023, or the total value that we could possibly get by reading an analog pin five times and adding all those values together. We could have divided `sensorValue` by 5 and then divided the result by 4, but the `map()` function gives us some options to adjust these reading later on. We end the `readSensor()` function with the `return` statement, as follows:

```
return sensorValue;
```

This line returns the value of `sensorValue` back to the calling function. Let's say we had the following statement in our `loop()` function:

```
int sensorValue = readSensor();
```

This line calls the `readSensor()` function and returns the final conditioned sensor reading back to the `loop()` function, assigning that value to the variable `sensorValue`. In this case, we have used the variable `sensorValue` twice but that's okay because if you remember, a function's scope is dependant on where it was declared. In each case here, we have declared them locally and so their scope does not extend beyond the function that they were declared in.

In addition to that, a value that is returned by a function is immediately forgotten by that function. We will need to do something with that returned value in our main `loop()` function or it will be lost to us.

Now we have made the assumption here that we want to use a function return to specifically return a value. We could also use the return keyword to immediately exit a function if a certain condition was met. The following is a hypothetical example:

```
if (sensorValue < 100) return;
```

Maybe we want to ignore readings that are below a certain threshold, so we could use the `return` keyword to exit a function without returning a value. Here we do that using an `if` statement with a condition that if true, will result in leaving the function and returning to normal program flow.

Function Parameters

In our last example we assumed that the analog pin being used was A0, but that might be a little shortsighted. Instead we can use **function parameters** to pass data like a pin number to the function, in this case to tell the function which pin to read from. Let's look at that function again, as follows: